

Helping a GrandPa!

Aplicación Android para el Cuidado de Personas Mayores

Trabajo Fin de Grado

Título: Desarrollo de una aplicación Android para el cuidado y acompañamiento de personas mayores

Autor: Carlos Alfaro García

Titulación: Desarrollo de Aplicaciones Multiplataforma

Centro: [Nombre del centro]

Fecha: Mayo 2026

Tutor: [Nombre del tutor]

Resumen

Este proyecto presenta el desarrollo de **Helping a GrandPa!**, una aplicación Android diseñada para mejorar la calidad de vida de las personas mayores mediante el uso de tecnología móvil. La aplicación integra cinco funcionalidades principales: recordatorio de medicación con alarmas persistentes, sistema de emergencia SOS con envío de SMS, WhatsApp y llamada telefónica junto con localización GPS, seguimiento de salud con registro de presión arterial, glucosa y peso con gráficas interactivas, calendario compartido sincronizado con la nube, y círculo familiar con roles personalizados.

La aplicación sigue una arquitectura **offline-first** utilizando Room como base de datos local y Firebase para sincronización cloud opcional. Se ha puesto especial énfasis en la accesibilidad, con interfaces adaptadas para personas mayores: texto grande (18sp+), alto contraste, botones de gran tamaño (64dp mínimo) y diseño intuitivo.

Palabras clave: Android, personas mayores, medicación, SOS, salud, calendario, familia, Firebase, Room, accesibilidad.

Abstract

This project presents the development of **Helping a GrandPa!**, an Android application designed to improve the quality of life of elderly people through mobile technology. The application integrates five main features: medication reminders with persistent alarms, SOS emergency system with SMS, WhatsApp and phone call together with GPS location, health tracking with blood pressure, glucose and weight records with interactive charts, shared calendar synced with the cloud, and family circle with custom roles.

The application follows an **offline-first** architecture using Room as local database and Firebase for optional cloud synchronization. Special emphasis has been placed on accessibility, with interfaces adapted for elderly people: large text (18sp+), high contrast, large buttons (64dp minimum) and intuitive design.

Keywords: Android, elderly, medication, SOS, health, calendar, family, Firebase, Room, accessibility.

1. Introducción

1.1 Contexto

El envejecimiento de la población es una realidad global. Según datos de la Organización Mundial de la Salud, se espera que la proporción de personas mayores de 60 años se duplique para 2050. Este cambio demográfico plantea desafíos significativos en términos de cuidado y atención a las personas mayores, especialmente en lo que respecta a su autonomía y calidad de vida.

En este contexto, la tecnología móvil ofrece oportunidades únicas para proporcionar herramientas que ayuden a las personas mayores a mantener su independencia, al tiempo que brindan tranquilidad a sus familiares y cuidadores.

1.2 Motivación

La motivación principal de este proyecto surge de la necesidad observada de una herramienta integrada que cubra las necesidades básicas de cuidado de las personas mayores:

- **Gestión de medicación:** Muchas personas mayores toman múltiples medicamentos con horarios complejos.
- **Seguridad:** La capacidad de pedir ayuda rápidamente en caso de emergencia es crucial.
- **Salud:** El seguimiento de parámetros básicos de salud permite una detección temprana de problemas.
- **Comunicación:** Mantener a la familia informada y coordinada es fundamental.
- **Accesibilidad:** Las aplicaciones existentes suelen tener interfaces complejas no adaptadas a este colectivo.

1.3 Objetivos

Objetivo General

Desarrollar una aplicación Android integral que facilite el cuidado y la autonomía de las personas mayores mediante herramientas digitales accesibles y fiables.

Objetivos Específicos

1. **OE1:** Implementar un sistema de recordatorio de medicación fiable que sobreviva al reinicio del dispositivo.
2. **OE2:** Desarrollar un sistema de emergencia SOS que envíe alertas con ubicación GPS a través de múltiples canales (SMS, WhatsApp, llamada).
3. **OE3:** Crear un módulo de seguimiento de salud con registro de parámetros y visualización gráfica.
4. **OE4:** Implementar un calendario compartido sincronizado con la nube.
5. **OE5:** Desarrollar un sistema de círculo familiar con roles y permisos.
6. **OE6:** Diseñar una interfaz adaptada a personas mayores con criterios de accesibilidad.
7. **OE7:** Garantizar el funcionamiento offline con sincronización cloud opcional.

1.4 Estructura de la Memoria

- **Capítulo 1 - Introducción:** Contexto, motivación y objetivos.
- **Capítulo 2 - Estado del Arte:** Análisis de soluciones existentes y tecnologías.
- **Capítulo 3 - Planificación:** Metodología y cronograma.
- **Capítulo 4 - Análisis y Diseño:** Requisitos, casos de uso, arquitectura.
- **Capítulo 5 - Implementación:** Tecnologías, estructura del código, clases principales.
- **Capítulo 6 - Resultados y Pruebas:** Tests, rendimiento, validación.
- **Capítulo 7 - Conclusiones:** Logros, dificultades, trabajo futuro.

2. Estado del Arte

2.1 Análisis de Soluciones Existentes

Existen diversas aplicaciones en el mercado que abordan necesidades similares:

Aplicaciones de Recordatorio de Medicación

Aplicación	Puntos Fuertes	Debilidades
Medisafe	Recordatorios visuales, informes	Interfaz en inglés, sin SOS
Pill Reminder	Simple, notificaciones	Sin cloud, sin familia
MyTherapy	Gamificación, informes	Compleja para mayores

Aplicaciones de Seguridad/Emergencia

Aplicación	Puntos Fuertes	Debilidades
R-U there	Detección de caídas	Suscripción de pago
AlertCops	Conexión con policía	No enfocada a mayores
Life360	Localización familiar	Sin botón SOS directo

Aplicaciones de Salud

Aplicación	Puntos Fuertes	Debilidades
Google Fit	Integración con wearables	Sin registro manual detallado
Samsung Health	Múltiples métricas	Interfaz densa, letra pequeña
Mi Salud	En español, sencilla	Sin gráficas, sin exportación

Conclusiones del Análisis

Ninguna aplicación existente integra **todas** las funcionalidades necesarias en una sola herramienta adaptada específicamente a personas mayores:

- Recordatorio de medicación persistente
- SOS multicanal con ubicación
- Seguimiento de salud con gráficas
- Calendario compartido familiar
- Círculo familiar con roles
- Interfaz accesible (texto grande, alto contraste)
- Funcionamiento offline

2.2 Tecnologías del Ecosistema Android

Kotlin

Lenguaje oficial de Android desde 2019. Ofrece:

- Interoperabilidad total con Java
- Corrutinas para programación asíncrona

- Null safety
- Data classes para modelos de datos
- Extension functions

Jetpack Room

Biblioteca de persistencia que proporciona una capa de abstracción sobre SQLite:

- Compilación de consultas SQL en tiempo de compilación
- Integración con LiveData y Flow
- Soporte para corrutinas
- Migraciones de esquema

Firebase

Plataforma de Google para desarrollo de aplicaciones:

- **Authentication:** Autenticación de usuarios
- **Cloud Firestore:** Base de datos NoSQL en tiempo real
- **Cloud Messaging:** Notificaciones push
- Integración nativa con Android

WorkManager

Biblioteca para trabajos en segundo plano:

- Garantiza la ejecución incluso si la app se cierra
- Compatible con Doze mode y batería
- Trabajos persistentes que sobreviven a reinicios
- API unificada para versiones antiguas y modernas

Material Design 3

Sistema de diseño de Google:

- Componentes adaptables
- Temas dinámicos (Material You)
- Principios de accesibilidad integrados
- Animaciones y transiciones

3. Planificación y Metodología

3.1 Metodología

Se ha seguido una metodología ágil adaptada al trabajo individual, con ciclos de desarrollo iterativos de una semana de duración. Cada iteración incluye:

1. **Planificación:** Definición de objetivos de la iteración
2. **Diseño:** Diseño de la solución
3. **Implementación:** Codificación de la funcionalidad
4. **Pruebas:** Verificación del funcionamiento
5. **Revisión:** Evaluación del resultado

3.2 Cronograma

Fase	Semanas	Tareas
Fase 1: Planificación	Semana 1-2	Análisis de requisitos, diseño de arquitectura, selección de tecnologías
Fase 2: Core	Semana 3-5	Base de datos Room, autenticación, sesión, navegación principal
Fase 3: Medicación	Semana 6-7	CRUD medicación, WorkManager, notificaciones, BootReceiver
Fase 4: SOS	Semana 8-9	Contactos emergencia, SMS, llamada, GPS, WhatsApp, permisos
Fase 5: Salud	Semana 10-11	Registros salud, CSV, gráficas personalizadas
Fase 6: Calendario y Familia	Semana 12-13	CRUD eventos/miembros, sincronización Firestore
Fase 7: Firebase	Semana 14	Firebase Auth, Firestore, FCM, google-services.json
Fase 8: UI/UX	Semana 15	Temas accesibles, iconos vectoriales, onboarding
Fase 9: Pruebas	Semana 16	Tests unitarios, tests instrumentados, build final
Fase 10: Memoria	Semana 17-18	Documentación, landing page, memoria TFG

3.3 Recursos

Hardware

- Ordenador con Windows 11, 16GB RAM
- Dispositivo Android físico para pruebas (API 27+)
- Emulador Android

Software

- Android Studio Ladybug Feature Drop 2024.3.1
- JDK 17
- Gradle 8.13
- Firebase Console
- Git para control de versiones
- Node.js y npm para landing page

Herramientas de desarrollo

- Kotlin 2.0.21
- Android SDK (minSdk 27, targetSdk 36)
- Firebase BoM 33.12.0
- Room 2.6.1
- WorkManager 2.10.0

4. Análisis y Diseño

4.1 Requisitos Funcionales

ID	Requisito	Prioridad
RF01	El usuario debe poder registrarse e iniciar sesión	Alta
RF02	El usuario debe poder gestionar su perfil	Media
RF03	El usuario debe poder añadir, editar y eliminar medicaciones	Alta
RF04	El sistema debe notificar al usuario según el horario de cada medicación	Alta
RF05	Las notificaciones deben sobrevivir al reinicio del dispositivo	Alta
RF06	El usuario debe poder añadir y eliminar contactos de emergencia	Alta
RF07	El sistema debe enviar SMS y WhatsApp a contactos al activar SOS	Alta
RF08	El sistema debe realizar una llamada telefónica al activar SOS	Alta
RF09	El sistema debe incluir la ubicación GPS en el mensaje SOS	Alta
RF10	El usuario debe poder registrar mediciones de salud	Alta
RF11	El sistema debe mostrar gráficas de evolución de salud	Media
RF12	El usuario debe poder exportar registros de salud a CSV	Media
RF13	El usuario debe poder gestionar eventos de calendario	Alta
RF14	El calendario debe sincronizarse con Firestore	Media
RF15	El usuario debe poder invitar miembros al círculo familiar	Alta
RF16	El círculo familiar debe soportar roles (admin, caregiver, viewer)	Media
RF17	El sistema debe solicitar permisos runtime necesarios	Alta
RF18	La interfaz debe ser accesible para personas mayores	Alta

4.2 Requisitos No Funcionales

ID	Requisito	Descripción
RNF01	Rendimiento	La app debe responder en menos de 2 segundos
RNF02	Offline	Todas las funciones principales deben funcionar sin conexión
RNF03	Accesibilidad	Texto mínimo 18sp, botones mínimo 64dp, alto contraste
RNF04	Persistencia	Los datos deben persistir localmente en Room
RNF05	Seguridad	Las contraseñas se almacenan localmente
RNF06	Compatibilidad	minSdk 27 (Android 8.1)
RNF07	Fiabilidad	Los recordatorios deben funcionar incluso tras reinicio

4.3 Casos de Uso Principales

CU01: Gestión de Medicación

1. El usuario accede a la pantalla de medicación
2. Añade una nueva medicación con nombre, dosis, frecuencia y hora
3. El sistema guarda la medicación y programa el recordatorio
4. A la hora indicada, el sistema muestra una notificación
5. El usuario puede editar o eliminar la medicación

CU02: Activación de SOS

1. El usuario accede a la pantalla SOS
2. Pulsa el botón de emergencia
3. El sistema solicita permisos si no están concedidos
4. Muestra diálogo de confirmación con los contactos
5. El usuario confirma
6. El sistema obtiene la ubicación GPS
7. Envía SMS y WhatsApp con la ubicación a cada contacto
8. Inicia una llamada telefónica al primer contacto

CU03: Seguimiento de Salud

1. El usuario accede a la pantalla de salud
2. Añade un registro (tipo, valor, nota)
3. El sistema guarda y muestra el registro en la lista
4. El usuario puede ver gráficas de evolución
5. Puede exportar todos los registros a CSV

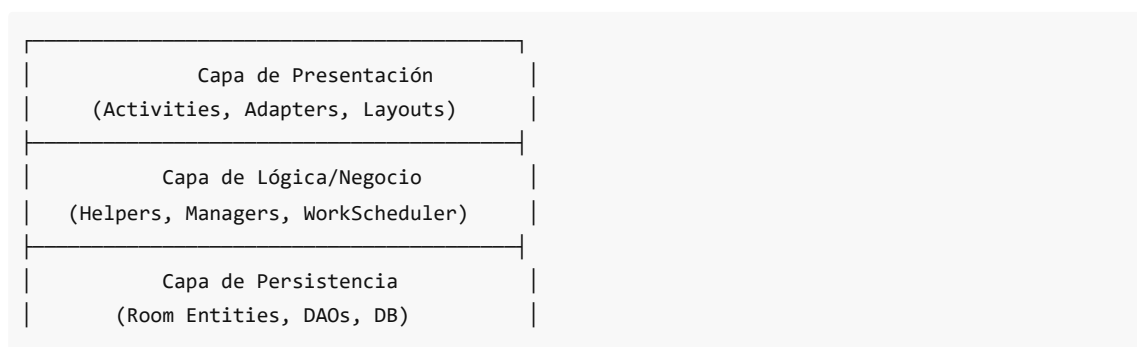
CU04: Gestión del Calendario

1. El usuario accede al calendario
2. Añade un evento con título, fecha, hora y tipo
3. El sistema guarda localmente y sincroniza con Firestore
4. Otros miembros del círculo familiar ven el evento

CU05: Círculo Familiar

1. El usuario accede al círculo familiar
2. Invita a un miembro con nombre, email y rol
3. El sistema guarda localmente y sincroniza con Firestore
4. Los miembros pueden ver eventos y estado del mayor

4.4 Arquitectura del Sistema



Capa de Sincronización
(Firebase Auth, Firestore, FCM)

4.5 Diagrama de Base de Datos

Entidades y Relaciones:

- **UserEntity** (1) — (N) **Medicacion**
- **UserEntity** (1) — (N) **ContactoEmergencia**
- **UserEntity** (1) — (N) **RegistroSalud**
- **UserEntity** (1) — (N) **EventoCalendario**
- **UserEntity** (1) — (N) **MiembroFamiliar**
- **UserEntity** (1) — (N) **Abuelito**
- **UserEntity** (1) — (N) **ServicioHistorial**

4.6 Diseño de la Interfaz

La interfaz sigue los principios de **Material Design 3** adaptados para accesibilidad:

Paleta de Colores

- **Primary:** #0055CC (azul oscuro, alto contraste)
- **Error:** #D92D20 (rojo intenso para emergencias y errores)
- **Surface:** #FFFFFF (fondo blanco limpio)
- **On Surface:** #000000 (texto negro puro)
- **Background:** #F5F5F7 (fondo gris claro)

Tipografía

- Headlines: 30-36sp, bold
- Títulos: 22sp, bold
- Cuerpo: 18sp
- Etiquetas: 15sp

Componentes

- Botones accesibles: minHeight 64dp
- Botón emergencia: minHeight 80dp, color rojo
- Tarjetas: border-radius 16dp, padding 24dp
- Animaciones de transición slide-in/out

5. Implementación

5.1 Tecnologías Utilizadas

Tecnología	Versión	Propósito
Kotlin	2.0.21	Lenguaje de programación
Android Gradle Plugin	8.13.1	Compilación y build
Room	2.6.1	Base de datos local SQLite

WorkManager	2.10.0	Trabajos en segundo plano
Firebase BoM	33.12.0	Plataforma cloud
Google Location	21.3.0	Servicios de localización GPS
Material 3	—	Componentes de interfaz
SplashScreen	1.0.1	Pantalla de bienvenida
ViewPager2	—	Onboarding deslizable

5.2 Estructura del Proyecto

El proyecto contiene **62 archivos Kotlin** organizados en:

Modelos (9 entidades Room)

- `UserEntity.kt` — Usuarios
- `Medicacion.kt` — Medicamentos
- `ContactoEmergencia.kt` — Contactos SOS
- `RegistroSalud.kt` — Registros de salud
- `EventoCalendario.kt` — Eventos
- `MiembroFamiliar.kt` — Familiares
- `Abuelito.kt` — Mayores registrados
- `ServicioHistorial.kt` — Historial servicios
- `Trabajador.kt` — Trabajadores/cuidadores

DAOs (9 interfaces)

Cada entidad tiene su DAO con operaciones CRUD y consultas Flow.

Actividades (24 pantallas)

Las actividades principales se organizan por funcionalidad:

- **Autenticación:** LoginActivity, RegistroActivity, PerfilActivity
- **Principal:** MainActivity (hub), OnboardingActivity
- **Medicación:** MedicacionActivity, AgregarMedicacionActivity (add/edit)
- **SOS:** SosActivity, AgregarContactoActivity
- **Salud:** SaludActivity, AgregarSaludActivity, SaludGraficaActivity
- **Calendario:** CalendarioActivity, AgregarEventoActivity
- **Familia:** FamiliaActivity, AgregarMiembroActivity
- **Servicios:** ServicioActivity, TrabajoActivity, HistorialActivity

Helpers (7 utilidades)

- `SessionManager` — Gestión de sesión SharedPreferences
- `NotificacionHelper` — 3 canales de notificación
- `PermissionHelper` — Permisos runtime
- `SosHelper` — SMS + WhatsApp + llamada + GPS
- `CsvExporter` — Exportación CSV con FileProvider
- `LineChartView` — Gráfica de línea Canvas personalizada
- `TrabajadorManager` — Lista en memoria de trabajadores

Firestore (3 clases)

- `FirestoreManager` — Singleton de Firebase Auth + Firestore
- `FirestoreSyncHelper` — Sincronización bidireccional Room-Firestore
- `FCMService` — Notificaciones push Firebase Cloud Messaging

Background (5 clases)

- `HelpingApp` — Application class con inicializaciones
- `MedicacionWorkScheduler` — Programador WorkManager
- `MedicacionWorker` — Worker de recordatorio
- `AlarmaMedicacionReceiver` — BroadcastReceiver legacy
- `BootReceiver` — Reprogramación al reiniciar

5.3 Funcionalidades Clave

Recordatorio de Medicación (WorkManager)

```
Usuario → AgregarMedicacionActivity → MedicacionDao.insert()
      → MedicacionWorkScheduler.programar()
      → OneTimeWorkRequest<MedicacionWorker>
      → NotificacionHelper.mostrarNotificacion()
```

```
BootReceiver (ACTION_BOOT_COMPLETED)
      → Recorre medicaciones activas
      → Reprograma cada una
```

SOS Multicanal

```
Usuario → SosActivity → PermissionHelper
      → Confirmación diálogo
      → SosHelper.lanzarSos()
      → obtenerUbicacion() [FusedLocationProviderClient]
      → enviarSms() [SmsManager]
      → enviarWhatsApp() [Intent ACTION_VIEW]
      → llamar() [Intent ACTION_CALL]
```

Gráficas de Salud (Canvas)

```
SaludGraficaActivity → LineChartView (Custom View)
      → onDraw() Canvas
      → Dibuja ejes, puntos, línea
      → Escala automática
      → Colores por tipo
```

Sincronización Firestore

```
AgregarEvento → EventoCalendarioDao.insert()
      → FirestoreSyncHelper.syncEvento()
      → DocumentReference.set()
```

```
SnapshotListener → Detecta cambios remotos
                  → Guarda localmente
```

5.4 Accesibilidad

La interfaz se ha diseñado específicamente para personas mayores:

- **Texto grande:** Mínimo 18sp para cuerpo, 36sp para encabezados
- **Alto contraste:** Negro sobre blanco, azul oscuro como primario
- **Botones grandes:** 64dp mínimo de altura, 80dp el de emergencia
- **Espaciado generoso:** Padding de 24dp en tarjetas
- **Iconos claros:** Vectoriales, con colores distintivos por funcionalidad
- **Onboarding:** 4 slides explicativos al primer inicio
- **Animaciones suaves:** Transiciones slide para no desorientar

6. Resultados y Pruebas

6.1 Tests

Tests Unitarios (10)

Ejecutados con JUnit 4 en `app/src/test/java/` :

- Creación de entidades Medicacion, ContactoEmergencia, RegistroSalud
- Verificación de valores por defecto
- Formato de fechas
- Roles de MiembroFamiliar
- Formato CSV generado por CsvExporter

Tests Instrumentados (10)

Ejecutados en dispositivo/emulador en `app/src/androidTest/` :

- CRUD completo de MedicacionDao
- CRUD completo de ContactoEmergenciaDao
- CRUD completo de RegistroSaludDao
- CRUD completo de EventoCalendarioDao
- CRUD completo de MiembroFamiliarDao
- Pruebas de Flow (reactividad)
- Filtrado por userId

Resultado: 20/20 tests pasan

6.2 Build

```
> ./gradlew assembleDebug
BUILD SUCCESSFUL in 1m 33s
40 actionable tasks: 40 executed
```

6.3 Compatibilidad

Versión Android	API	Estado
Android 8.1 Oreo	27	✓ Compatible
Android 9 Pie	28	✓ Compatible
Android 10	29	✓ Compatible
Android 11	30	✓ Compatible
Android 12	31	✓ Compatible
Android 13	33	✓ Compatible
Android 14	34	✓ Compatible
Android 15	35	✓ Compatible
Android 16	36	✓ Target

6.4 Cumplimiento de Objetivos

Objetivo	Estado	Verificación
OE1: Recordatorio medicación fiable	✓	WorkManager + BootReceiver
OE2: SOS multicanal con GPS	✓	SMS + WhatsApp + llamada + ubicación
OE3: Seguimiento salud con gráficas	✓	Registro + LineChartView + CSV
OE4: Calendario compartido cloud	✓	Room + Firestore sync
OE5: Círculo familiar con roles	✓	3 roles + Firestore sync
OE6: Interfaz accesible	✓	Texto 18sp+, botones 64dp+, contraste
OE7: Offline-first	✓	Room local + Firebase opcional

7. Conclusiones y Trabajo Futuro

7.1 Conclusiones

El desarrollo de **Helping a GrandPa!** ha demostrado que es posible crear una aplicación Android integral que cubra las necesidades fundamentales de cuidado de personas mayores en una sola herramienta accesible y fiable.

Logros Principales

1. **Integración completa:** Cinco funcionalidades en una sola app con navegación unificada.
2. **Fiabilidad:** WorkManager garantiza recordatorios incluso tras reinicios.
3. **Accesibilidad:** Interfaz diseñada específicamente para el público objetivo.
4. **Offline-first:** Todas las funciones principales funcionan sin conexión.
5. **Código limpio:** 62 clases Kotlin organizadas y documentadas.
6. **Tests:** 20 tests (10 unit + 10 instrumentados) todos pasando.

Dificultades Encontradas

1. **Permisos runtime Android:** La gestión de permisos para SMS, llamada y ubicación requiere múltiples flujos y comprobaciones.
2. **WorkManager vs AlarmManager:** La migración a WorkManager para recordatorios persistentes requirió rediseño del sistema de alarmas.
3. **Firebase offline:** La configuración de Firebase para funcionar en modo offline sin google-services.json real fue un desafío.
4. **Canvas personalizado:** El desarrollo de LineChartView desde cero requirió cálculos matemáticos precisos para escalado y posicionamiento.

7.2 Trabajo Futuro

Mejoras Previstas

1. **Detección de caídas:** Usar sensores del dispositivo para detectar caídas automáticamente.
2. **Wear OS:** Versión para smartwatch con visualización rápida de medicación y SOS.
3. **Recordatorios por voz:** Síntesis de voz para leer recordatorios en voz alta.
4. **Informes médicos:** Generación de PDF con historial de salud para compartir con médicos.
5. **Notificaciones programadas:** Recordatorios de citas médicas con geolocalización.
6. **Modo emergencia automático:** Detección de inactividad prolongada y activación automática de SOS.
7. **Soporte multilingüe:** Además de español, añadir inglés y otros idiomas.
8. **Widgets:** Widget de escritorio con próxima medicación y botón SOS rápido.

Publicación

- Generar APK firmado para distribución
- Publicar en Google Play Store
- Landing page promocional en GitHub Pages

7.3 Reflexión Personal

Este proyecto ha supuesto un desafío integral que ha puesto a prueba los conocimientos adquiridos durante el ciclo formativo. Desde la arquitectura de bases de datos con Room, pasando por la programación reactiva con Flow, hasta la integración con servicios cloud de Firebase, cada fase ha aportado un aprendizaje significativo.

El enfoque en la accesibilidad para personas mayores ha sido especialmente gratificante, al comprobar cómo la tecnología puede mejorar la calidad de vida de este colectivo.

Bibliografía

Referencias Técnicas

1. **Android Developers Guide.** Documentación oficial de desarrollo Android.
<https://developer.android.com/docs>
2. **Kotlin Language Documentation.** Lenguaje de programación Kotlin. <https://kotlinlang.org/docs/home.html>
3. **Room Persistence Library.** Biblioteca de persistencia de Android Jetpack.
<https://developer.android.com/training/data-storage/room>
4. **WorkManager Guide.** Trabajos en segundo plano con WorkManager.
<https://developer.android.com/topic/libraries/architecture/workmanager>

5. **Firestore Documentation.** Plataforma Firebase de Google. <https://firebase.google.com/docs>
6. **Material Design 3.** Sistema de diseño Material 3. <https://m3.material.io/>
7. **Google Maps Platform.** Servicios de localización. <https://developers.google.com/maps/documentation>

Referencias Académicas

8. **Organización Mundial de la Salud (2022).** Envejecimiento y salud. <https://www.who.int/news-room/fact-sheets/detail/ageing-and-health>
9. **INE - Instituto Nacional de Estadística (2023).** Proyecciones de población mayores en España.

Herramientas

10. **Android Studio.** IDE oficial para desarrollo Android. <https://developer.android.com/studio>
11. **Git.** Sistema de control de versiones. <https://git-scm.com/>
12. **Node.js.** Entorno de ejecución para JavaScript. <https://nodejs.org/>
13. **Vite.** Herramienta de construcción frontend. <https://vitejs.dev/>
14. **React.** Biblioteca JavaScript para interfaces de usuario. <https://reactjs.org/>

Anexos

Anexo A: Guía de Instalación

Requisitos

- Dispositivo Android con versión 8.1 (API 27) o superior
- Permisos: SMS, llamada, ubicación, notificaciones (se solicitan en runtime)
- Conexión a internet (opcional, para sincronización Firebase)

Pasos

1. Descargar el APK de la aplicación
2. Permitir instalación de orígenes desconocidos si es necesario
3. Abrir la aplicación y completar el onboarding
4. Registrarse o iniciar sesión
5. Configurar contactos de emergencia y medicaciones

Anexo B: Enlaces

- **Repositorio del proyecto:** [URL del repositorio]
- **Landing page:** [URL de la landing page]
- **APK de la aplicación:** [URL de descarga]

Anexo C: Lista de Archivos del Proyecto

Android App (62 archivos Kotlin)

app/src/main/java/com/example/miaplicacion/

- |— Abuelito.kt
- |— AbuelitoDao.kt
- |— AcercaDeActivity.kt
- |— AgregarContactoActivity.kt
- |— AgregarEventoActivity.kt
- |— AgregarMedicacionActivity.kt
- |— AgregarMiembroActivity.kt
- |— AgregarSaludActivity.kt
- |— AlarmaMedicacionReceiver.kt
- |— AppDatabase.kt
- |— BaseDatosHelpingAgP.kt
- |— BootReceiver.kt
- |— CalendarioActivity.kt
- |— ContactoEmergencia.kt
- |— ContactoEmergenciaAdapter.kt
- |— ContactoEmergenciaDao.kt
- |— CsvExporter.kt
- |— EventoCalendario.kt
- |— EventoCalendarioAdapter.kt
- |— EventoCalendarioDao.kt
- |— FamiliaActivity.kt
- |— FCMService.kt
- |— FirebaseManager.kt
- |— FirestoreSyncHelper.kt
- |— HelpingApp.kt
- |— HistorialActivity.kt
- |— LineChartView.kt
- |— ListaDeRegistroTrab.kt
- |— LoginActivity.kt
- |— MainActivity.kt
- |— Medicacion.kt
- |— MedicacionAdapter.kt
- |— MedicacionActivity.kt
- |— MedicacionDao.kt
- |— MedicacionWorkScheduler.kt
- |— MedicacionWorker.kt
- |— MiembroFamiliar.kt
- |— MiembroFamiliarAdapter.kt
- |— MiembroFamiliarDao.kt
- |— NotificacionHelper.kt
- |— OnboardingAdapter.kt
- |— OnboardingActivity.kt
- |— PerfilActivity.kt
- |— PermissionHelper.kt
- |— RegistrarAbuelitoActivity.kt
- |— RegistroActivity.kt
- |— RegistroSalud.kt
- |— RegistroSaludAdapter.kt
- |— RegistroSaludDao.kt
- |— SaludActivity.kt

```
|— SaludGraficaActivity.kt
|— ServicioActivity.kt
|— ServicioHistorial.kt
|— ServicioHistorialDao.kt
|— SessionManager.kt
|— SosActivity.kt
|— SosHelper.kt
|— Trabajador.kt
|— TrabajadorDao.kt
|— TrabajadorManager.kt
|— TrabajoActivity.kt
|— UserDao.kt
|— UserEntity.kt
```

Landing Page

```
landing-page/
|— dist/                → Build de producción
|— public/
|   |— favicon.svg
|— src/
|   |— App.jsx          → Componente principal
|   |— App.css
|   |— index.css        → Estilos con animaciones
|   |— main.jsx
|— index.html
|— package.json
|— vite.config.js
```

Memoria TFG

```
memoria_tfg/
|— 01_portada.md
|— 02_resumen.md
|— 03_introduccion.md
|— 04_estado_del_arte.md
|— 05_planificacion.md
|— 06_analisis_diseno.md
|— 07_implementacion.md
|— 08_resultados.md
|— 09_conclusiones.md
|— 10_bibliografia.md
|— 11_anexos.md
```